

AA 272 Fall 2025. Homework 2

YOUR NAME, YOUR_EMAIL@stanford.edu

Instructors: Prof. Grace Gao, Keidai Iiyama, Guillem Casadesus Vila, and Hannah Nabavi

October 3, 2025

Goal of this problem set: In week 2, we learned about the navigation equations and an iterative approach to solving these equations. In this problem set, we will understand why the community uses *weighted* least squares as the standard baseline for GPS and GNSS. To that end, we will investigate least squares practically, theoretically, and geometrically.

Instructions & Notes

1. This problem set is **due at 4:00 PM on October 17, 2025 via Gradescope**.
2. This homework is scored out of **50 points total** and includes a corresponding self-care component. The point value for each homework question is indicated next to the question number. Note that each homework will be equally weighted in the calculation of your final grade.
3. Please note that homework is intended to supplement and reinforce your learning. We design them to guide you through the process of going deeper into the concepts we cover in class, and beyond.
4. You are free to discuss class contents and these problems with others. However, your submitted solutions and code must be your own work, written from scratch.
5. **AI tools** (e.g., Chat GPT, Gemini) can be viewed as search engines or even classmates. They must not be used to directly solve any of the problems. For example, it is acceptable to discuss general topics about GNSS, edit a plot, or how to solve a linear system using `numpy`.
6. **Ensure that you typeset your solutions.** We encourage using the provided notebooks or LaTeX to type up your solutions, but you are free to use Microsoft Word or other editors.
7. Ensure that your solution includes the plots required by the question, **in the specified format** outlined in the problem for full credit. **Please submit your code in the programming assignment entry on Gradescope.** You can submit Colab notebooks and other interactive formats as links in a `.txt` format.
8. If you are spending an excessive amount of time on any part of any question, please post on Ed or come to office hours (see Canvas).
9. Please **start early** so you can address any questions you might have, and have fun!

Problem 1: Solving the Navigation Equations with Newton-Raphson [26 points + 2 bonus]

Goal of this problem: Students will use GNSS measurements collected on an Android phone to perform localization using the Newton-Raphson localization algorithm introduced in Lecture 4.

For this problem, we will use highway data from three different time steps of the Google Smartphone Decimeter Challenge. We have provided three CSV files (`hw2_p1_dataset1_navdata.csv`, `hw2_p1_dataset2_navdata.csv`, `hw2_p1_dataset3_navdata.csv`) on Canvas with the relevant data. We will give you the least squares solution for the first data set (akin to a training data set), but we will not give you the least squares solution for the other data sets.

The columns in the data are as follows:

1. **gps millis:** This is the GPS millis time of the data. Each dataset has a different GPS millis. You do not need to use it for this problem. But, you would need to separate data by gps millis if you were running a whole trajectory (e.g., for your final project).
2. **gnss_id:** This is the GNSS constellation for this data. We will use all constellations in this assignment.
3. **sv_id:** The identifier for the satellite within a constellation. We will use all of the available satellites in this assignment. Satellites are repeated if they transmit in multiple signal bands¹. Still include all entries.
4. **corr prm:** This is the **corrected pseudorange** in meters. Specifically, we have taken the raw pseudorange and removed the satellite clock bias $B^{(k)}$, the inter-signal bias (i.e., inter-constellation) $b_{inter-signal}^{(k)}$, the estimated ionosphere bias $I^{(k)}$, and the estimated troposphere bias $T^{(k)}$. Namely,

$$\rho_{corrected} = \rho_{raw} + B^{(k)} - b_{inter-signal}^{(k)} - I^{(k)} - T^{(k)}$$

Note that compensating for the inter-signal bias is necessary when using multiple constellations. You will learn more about these other biases in the later weeks. But, in effect, these corrections clean the pseudorange measurements. As you will see in problem 2, compensating for these biases is important for solving the navigation equations, especially with least squares.

5. **x_svm:** The satellite x coordinate in ECEF in meters.
6. **y_svm:** The satellite y coordinate in ECEF in meters.
7. **z_svm:** The satellite z coordinate in ECEF in meters.

1.1 [15 points]

Follow the template from Lecture 4 to implement the **Newton-Raphson localization algorithm** for iterative least-squares optimization to solve the navigation equations. Your code should output (1) your optimized position, (2) your optimized clock bias, and (3) the number of iterations until it converges. Check that the optimized position and clock bias for the first data set are (close to) those listed below in ECEF. Also, note the number of iterations until it converged.

$$\mathbf{x} = [-2696209 \text{ m} \quad -4293890 \text{ m} \quad 3856549 \text{ m}]^T$$
$$b_u = 1832 \text{ m}$$

Hint: Below are some suggestions for the code structure. If you prefer object-oriented programming, you may find it easier to pass in the data set as a table-like data structure rather than passing separated arguments.

1. The top-level function (e.g., `newton_raphson(x0, bu_0, max_iterations)`) will take as input an initial guess for position `x0` and clock bias `bu_0`. To help with debugging and to solve the later problems, it will also take the input `max_iterations` which is the maximum number of iterations of Newton-Raphson. This top-level function will return the requisite outputs:
 - (1) your optimized position
 - (2) your optimized clock bias
 - (3) the number of iterations until it converges
2. Implement a function (e.g., `get_geometry_matrix(x_est, x_sv_m, y_sv_m, z_sv_m)`) which takes as input a set of satellite positions (`x_sv_m`, `y_sv_m`, `z_sv_m`) as well as the current position estimate (`x_est`) and outputs the geometry matrix G .
 - For each row of G , the first 3 columns make up a unit vector.
 - You can check the size/shape of G before returning to help debugging.
3. Implement a function (e.g., `get_theoretical_pseudoranges(x_est, b_est, x_sv_m, y_sv_m, z_sv_m)`) which takes as input the current position estimate (`x_est`), the current clock bias estimate (`b_est`), the satellite positions (`x_sv_m`, `y_sv_m`, `z_sv_m`).
 - *Note:* We do **not** need the satellite clock bias since the corrected pseudorange we provide in the data set has already corrected it.
 - This function should output the theoretical pseudorange measurement vector.
 - Again, you can check the size/shape of the theoretical pseudorange measurement vector before returning to help debugging.
4. Either as a function or in the top-level function (i.e., `newton_raphson(x0, bu_0, max_iterations)`), calculate the difference between the theoretical and measured pseudoranges.
5. Implement a function (e.g., `newton_raphson_step(G, delta_rho)`) which takes one step in the Newton-Raphson optimization. That is, it computes the update Δx and Δb_u .
6. Update the current position estimate and clock bias estimate as a function or in the top-level function (i.e., `newton_raphson(x0, bu_0, max_iterations)`).
 - In the top-level function, repeat the iteration until you either:
 1. have updates in Δx and Δb_u that are less than a centimeter, or
 2. hit the maximum number of iterations.

ANSWER

[Replace with your answer]

1.2 [2 points]

Starting from the Earth center with no clock bias (i.e., $\mathbf{x}_0 = \mathbf{0}$ and $b_{u,0} = 0$), what are your optimized positions and clock biases for the remaining data sets in ECEF? How many iterations were required until it converged?

ANSWER

[Replace with your answer]

1.3 [3 points]

Using an online converter or GNSS Lib Py, what is your **optimized position in LLA** (for each data set)? Add a screenshot of your position in Google Maps.

[Replace with your answer]

1.4 [3 points]

Try out different initial positions when initializing the Newton-Raphson algorithm. Use an initial condition on the other side of the planet (i.e., the **antipode** of the user location), and still using $b_{u,0} = 0$. What are your optimized positions and clock biases for the new initial conditions, for each data set? How many iterations were required until it converged?

ANSWER

[Replace with your answer]

1.5 [3 points]

Provide one example of an initial position (still using $b_{u,0} = 0$) where Newton-Raphson **does not converge in 20 iterations**. What is the estimated position and clock bias when it hits 20 iterations? Repeat for all three data sets.

Hint: You may need to try a point that is a few Earth radii away.

ANSWER

[Replace with your answer]

Bonus 1 [1 point]

The actual ground truth point for the first dataset is

Latitude: 37.44329905 degrees

Longitude: -122.12575639 degrees

This ground truth is a post-processed localization using a high-accuracy, high-precision GNSS receiver mounted on the vehicle. Use the Lecture 3 slides to estimate the **shift** between the least squares solution from Newton-Raphson to the ground truth. Report the value in degrees and in meters north (latitude) and east (longitude) from the least squares solution to the ground truth.

Hint: You can cross-check your solution with an online calculator (e.g., from Meridian Outpost)

ANSWER

[Replace with your answer]

Bonus 2 [1 point]

This shift between the least squares and the actual ground truth is consistent across all three data sets. What is causing this shift? How can we address this shift?

Hint: How long does it take for the satellite signal to reach us?

Hint: You do not need to implement your solution to address the shift to get credit, but you may find it useful to implement your solution to check your understanding.

ANSWER

[Replace with your answer]

Problem 2 (10 points): Theoretical Motivation for WLS

Goal of this problem: Students will learn about why we often use the *weighted* least-squares solution rather than the (ordinary) least-squares solution of the navigation equations.

What are the assumptions of this solution? Are these assumptions valid?

In Lecture 4, we derived the (ordinary) least-squares (OLS) solution to the linearized navigation equations as:

$$\delta \mathbf{y} = \begin{bmatrix} \delta \mathbf{x} \\ \delta b \end{bmatrix} = (G^\top G)^{-1} G^\top \delta \rho,$$

where $\delta \mathbf{y} \in \mathbb{R}^4$ is the correction to our state consisting of a position correction $\delta \mathbf{x} \in \mathbb{R}^3$ and a clock bias correction $\delta b \in \mathbb{R}$, $\delta \rho \in \mathbb{R}^N$ is the vector of pseudorange residuals for N visible satellites, and $G \in \mathbb{R}^{N \times 4}$ is the geometry matrix.

Ideally, we want the pseudorange residuals to be near zero (i.e., $\delta \rho \approx \mathbf{0}$), but we cannot achieve zero residuals in practice due to measurement errors. For example, the residuals can be 10 – 100 m in harsh urban environments!

In what sense is the (ordinary) least-squares solution (OLS) the *best* or *optimal* solution?

Building on the shoulders of giants, the work of Gauss (1777–1855) and Markov (1856–1922) proved that the OLS estimator is the *best linear unbiased estimator (BLUE)* under certain assumptions on the error.

First, what does BLUE mean?

- *Best* refers to the estimator with the minimum variance among all possible estimators with the restrictions and assumptions below.
- *Linear* refers to restricting to a linear model (i.e., $\delta \rho = G \delta \mathbf{y}$ is linear).
- *Unbiased* [1] refers to restricting the estimator to be centered around the true value if you repeat the estimation many times with new data.
- *Estimator* refers to the procedure to *estimate* an unknown quantity from data.

The assumptions on the error are quite important.

Specifically, this is the error ϵ in the linear model $\delta \rho = G \delta \mathbf{y} + \epsilon$.

The three assumptions [2] are below. The figure below illustrates these assumptions more graphically.

1. **Errors are uncorrelated.** The error $\epsilon^{(1)}$ from the pseudorange measurement of satellite 1 is not correlated with the error $\epsilon^{(2)}$ from the pseudorange measurement of satellite 2, and so on with the other satellites.
2. **Errors have equal variance** (AKA “spherical errors”). The errors $\epsilon^{(1)}$ from the pseudorange measurement in satellite 1 have a distribution with the same variances as the errors $\epsilon^{(2)}$ in satellite 2, and so on with the other satellites.

3. **Errors have an expected value (i.e., mean) of zero.** If we get many measurements from satellite 1, then the mean of the pseudorange residuals will be zero.

In practice, the second assumption is almost never true. Fortunately, Aitken (1895–1967) found a way to generalize least-squares estimation to remove the first and second assumptions. Aitken’s approach is called *generalized least-squares (GLS)* [3], but it requires good knowledge of the error distribution, which is quite difficult to achieve in practice. Moreover, approaches to estimate the correlation and variance between errors tend to be quite computationally heavy.

So, instead, we only drop the second assumption and try to account for the differences in variance between measurements, but we do not account for the correlation between errors. This approach is called **weighted least-squares (WLS)**. It turns out to be pretty good, which is why people still use it. We will see more about WLS in Lecture 6.

To provide intuition, we want to put a *larger* weight on satellites where the variance is *small* since these measurements provide a more precise understanding of the user location. Likewise, we want to put a *smaller* weight on satellites where the variance is *large* (Figure above).

To incorporate relative weighting of the pseudorange measurements, we will redefine our problem statement for the “best” $\delta \mathbf{y}$ solution to be:

$$\arg \min_{\delta \mathbf{y}} \|\delta \rho - G \delta \mathbf{y}\|_W^2,$$

where $W = \text{diag}(w_1, w_2, \dots, w_N)$ is a diagonal weighting matrix with positive weights w_i , and $\|\cdot\|_W$ is the Mahalanobis norm, defined as:

$$\|\mathbf{z}\|_W^2 = \mathbf{z}^\top W \mathbf{z},$$

where $\mathbf{z} \in \mathbb{R}^N$.

To provide intuition, the normal Euclidean distance (i.e., the Euclidean norm) arises if $W = I$, where I is the identity matrix. The contours of the Euclidean norm are circles whereas the contours of the Mahalanobis norm are ellipses.

[1]: The restriction on “unbiased” estimation is critical. There are biased estimators with lower variance. For example, “regularization” is a common bias that is introduced to lower variance, especially in a least-squares context. For more, I recommend AA 222 (Optimization) and CS/STATS 229 (Machine Learning).

[2]: Often, people incorrectly list an assumption or requirement that the errors *must* follow a multivariate Gaussian (i.e., a normal distribution). However, we do not *require* that the errors are distributed as a multivariate Gaussian. But, it turns out that we can make even stronger claims about least-squares estimation if the errors are Gaussian, as we assumed in Lecture 4. In many cases, GPS/GNSS errors can be well-modelled as Gaussians. For more, I recommend AA 273 (State Estimation) and CS/STATS 229 (Machine Learning).

[3]: Check out Chapter 6 of the GPS textbook and [Wikipedia: Generalized least squares](#).

2.1 [4 Points]

By following very similar steps as in Lecture 4, show that the solution to the WLS problem is:

$$\delta \mathbf{y} = (\mathbf{G}^\top \mathbf{W} \mathbf{G})^{-1} \mathbf{G}^\top \mathbf{W} \delta \rho$$

This is called the **weighted least-squares solution**.

ANSWER

[Replace with your answer]

Deliverable: Derivation of the provided equation starting at the WLS objective function. Please show all your work and justify your approach as necessary.

2.2 [2 Points]

Mathematically, why is it important for the diagonal weights of $\mathbf{W}$ to be **positive** for our defined problem statement and derivation?

Hint:

If you are stuck, head to Wolfram Alpha[4] and compare the output plots for each:

- Both positive weights: $\{x, y\} * \{1/2, 0\}, \{0, 1\} * \{x, y\}$
- One negative weight: $\{x, y\} * \{1/2, 0\}, \{0, -1\} * \{x, y\}$
- Different negative weight: $\{x, y\} * \{-1/2, 0\}, \{0, 1\} * \{x, y\}$
- Both negative weights: $\{x, y\} * \{-1/2, 0\}, \{0, -1\} * \{x, y\}$

[4]: Available at this link: <https://www.wolframalpha.com/>

ANSWER

[Replace with your answer]

Deliverable: Mathematical explanation (roughly 1 paragraph). Your response should reference the Mahalanobis norm definition.

2.3 [2 Points]

Why would some pseudorange measurements have better quality than others? In other words, why is assumption 2 (Errors have equal variance/spherical errors) violated in real measurements? Provide at least two concrete examples.

Hint: There are *many* possible answers. This problem set and lectures 1, 5, and 6 may provide some insights, but feel free to think outside the box!

ANSWER

[Replace with your answer]

Deliverable: At least two concrete examples and an explanation (roughly 1-3 sentences each) for why it leads to pseudorange measurements of different quality.

2.4 [2 Points]

Provide at least one concrete example where the errors are **correlated** (i.e., violating assumption 1), and at least one concrete example where the errors do not have **zero means** (i.e., violating assumption 3). *Note that weighted least-squares cannot handle these errors on its own.

Note: You may consider the corrections we applied in problem 1 and other insights from the problem 2 and 3 descriptions. However, there are *many* possible answers. Lectures 1, 5, and 6 may provide some insights, but feel free to think outside the box!

Hint: If you are stuck, think back to problem 2 on homework 1. From the skyplots, there are often satellites with similar elevation and azimuth in a multi-constellation setting. Are there errors that similarly affect satellites in nearby elevation and azimuth?

ANSWER

[Replace with your answer]

Deliverable: At least one concrete example for correlated errors and an explanation (roughly 1-3 sentences). At least one concrete example for errors without zero means (roughly 1-3 sentences).

Problem 3: Satellite Geometry [14 points + 3 bonus]

Goal of this problem: Students will learn about Dilution of Precision (DOP) in nominal and adverse settings.

Recall from Lecture 4 that $(G^\top G)^{-1}$ tells us how the user-satellite geometry affects our Dilution of Precision and state uncertainty, where G is the geometry matrix.

In class, we created the G matrix with unit direction vectors in the ECEF (Earth-Centered-Earth-Fixed) reference frame; however, it is often more intuitive and practical to understand our state uncertainty in our local reference frame, i.e., ENU frame (East-North-Up).

Thus, let us define a new geometry matrix $\tilde{G}$ as:

$$\tilde{G} = \begin{bmatrix} \left(-\mathbf{1}_{\text{ENU}}^{(1)}\right)^\top & 1 \\ \left(-\mathbf{1}_{\text{ENU}}^{(2)}\right)^\top & 1 \\ \vdots & \vdots \\ \left(-\mathbf{1}_{\text{ENU}}^{(N)}\right)^\top & 1 \end{bmatrix}$$

Notice, this matrix is almost identical to the one we derived in class, but where we use a unit direction vector that is defined in our local, ENU reference frame (denoted by $\mathbf{1}_{\text{ENU}}^{(1)}$).

Let us define our local DOP matrix as:

$$\tilde{H} = (\tilde{G}^\top \tilde{G})^{-1},$$

which corresponds to the following $\mathbb{R}^{4 \times 4}$ matrix:

$$\tilde{H} = \begin{bmatrix} \text{EDOP}^2 & \cdot & \cdot & \cdot \\ \cdot & \text{NDOP}^2 & \cdot & \cdot \\ \cdot & \cdot & \text{VDOP}^2 & \cdot \\ \cdot & \cdot & \cdot & \text{TDOP}^2 \end{bmatrix},$$

where EDOP is our East DOP, NDOP is our North DOP, VDOP is our Vertical DOP (i.e., up direction), and TDOP is the same as defined in lecture 4, i.e., our Time DOP.

Each of these terms refers to how much the satellite geometry *dilutes* (or degrades) our precision in the user position and time when the DOP value > 1 or improves our precision when the DOP value < 1 .

For example, a high EDOP and low NDOP mean that we have worse precision in the East direction and better precision in the North direction.

The off-diagonal terms are generally not zero, but will generally be smaller in magnitude¹.

With this matrix $\tilde{H}$, we additionally get the following definitions of Horizontal DOP (HDOP) and Position DOP (PDOP):

$$\text{HDOP} = \sqrt{\text{EDOP}^2 + \text{NDOP}^2}, \text{PDOP} = \sqrt{\text{EDOP}^2 + \text{NDOP}^2 + \text{VDOP}^2}.$$

3.1 [2 points]

Create a function called `get_unit_vector(az, el)`, which takes in a satellite's azimuth and elevation angle in degrees, `az` and `el`, respectively, and outputs the 3-dimensional unit vector which represents the “look” direction to that satellite in the local, ENU reference frame.

If you are using GNSS Lib Py, the function can be `get_unit_vector(navdata)`.

Hint: This [reference document](#) or chapter 6.1.2 of the GPS textbook might help in defining the unit direction vector in the local frame, given an azimuth and elevation angle.

Take a look at the section on *local horizon coordinates* in the linked file.

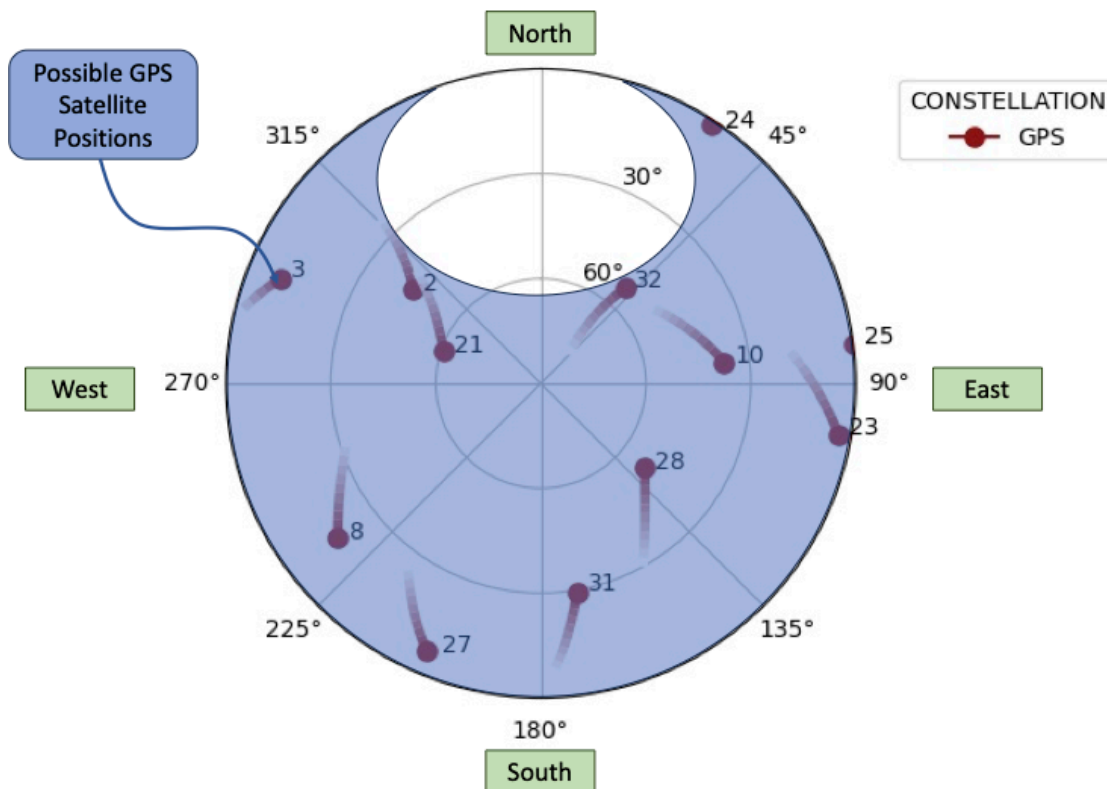
Hint: A unit vector (also, the output of your function!) has magnitude 1.

ANSWER

[Replace with your answer]

Using the skyplots from homework 1, recall that the GPS satellites can only appear in the blue subset of the skyplot in the figure below.

These satellite locations, along with their elevation and azimuth are included on Canvas as a CSV file named `hw2_p3_az_el_data.csv`



3.2 [1 point]

Using your function, `get_unit_vector`, what is the ENU direction vector to **GPS satellite 27**? (Azimuth: 203.0°, Elevation: 6.7°) You can intuitively check your answer by referencing the skyplot.

ANSWER

[Replace with your answer]

3.3 [2 points]

Create a function called `compute_dop_matrix(az_arr, el_arr)` (or if using GNSS Lib Py, `compute_dop_matrix(navdata)`) which takes in an array of satellite azimuth and elevation angles in degrees, `az_arr` and `el_arr`, respectively, and calls your `get_unit_vector` function from part (3.1) to output the DOP matrix $\tilde{\mathbf{H}}$ in the ENU frame.

Plot the resulting DOP matrix for the open sky condition as a heatmap.

Hint: Use the following function for heatmap generation.

```
[5]: def plot_dop_matrix(H, cmap='RdBu_r', vmin=-20, vmax=20,
                        symmetric_color=True, label_nums=True):
    if symmetric_color:
        if (vmin is not None) or (vmax is not None):
            if not np.isclose(np.abs(vmin), np.abs(vmax)):
                raise ValueError(f"Cannot force symmetric color since vmin_
↪({vmin}) is not of equal magnitude to vmax ({vmax})")
            else:
                # Estimate the vmin and vmax from values
                vmax = np.max(np.abs(H))
                vmin = - vmax

        # Plot the matrix as an image
        plt.imshow(H, cmap=cmap, vmin=vmin, vmax=vmax)
        plt.colorbar()

        # Set the tick marks
        ticks = [0, 1, 2, 3]
        tick_labels = ["East", "North", "Up", "Time"]
        plt.xticks(ticks, tick_labels)
        plt.yticks(ticks, tick_labels)

        # Label the numbers
        if label_nums:
            for (j,i), label in np.ndenumerate(H):
                plt.text(i,j, f"{label:.3f}", ha='center', va='center')
```

ANSWER

[Replace with your answer]

Deliverable: The resulting DOP matrix $\tilde{\mathbf{H}}$ (as a heatmap plot).

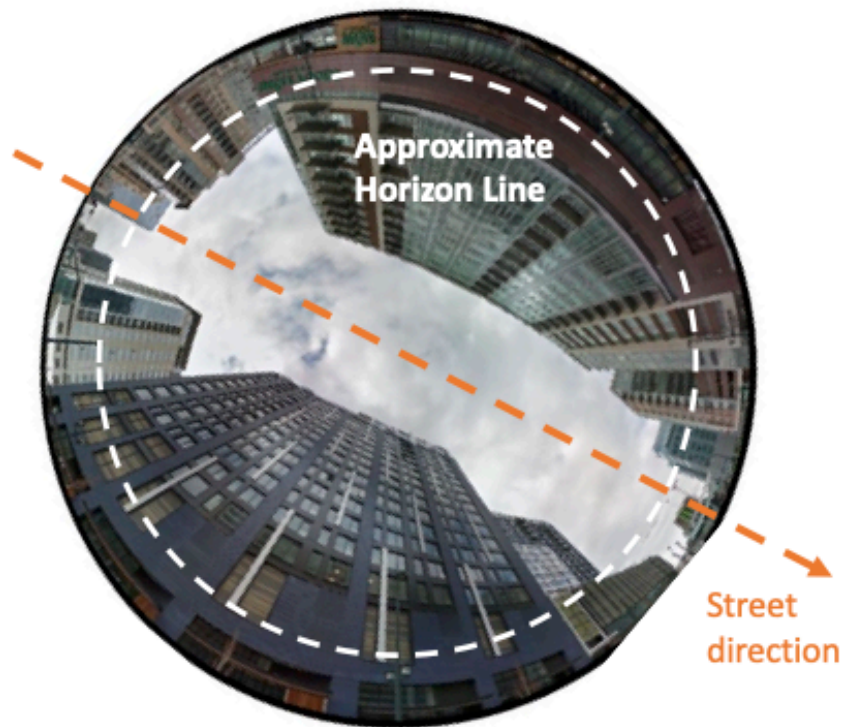
3.4 [2 points]

Create a function called `compute_pdop(H)`, which uses the output of your `compute_dop_matrix` function and computes the **PDOP** for the provided satellites. Check that the PDOP of the provided satellites is **1.29**.

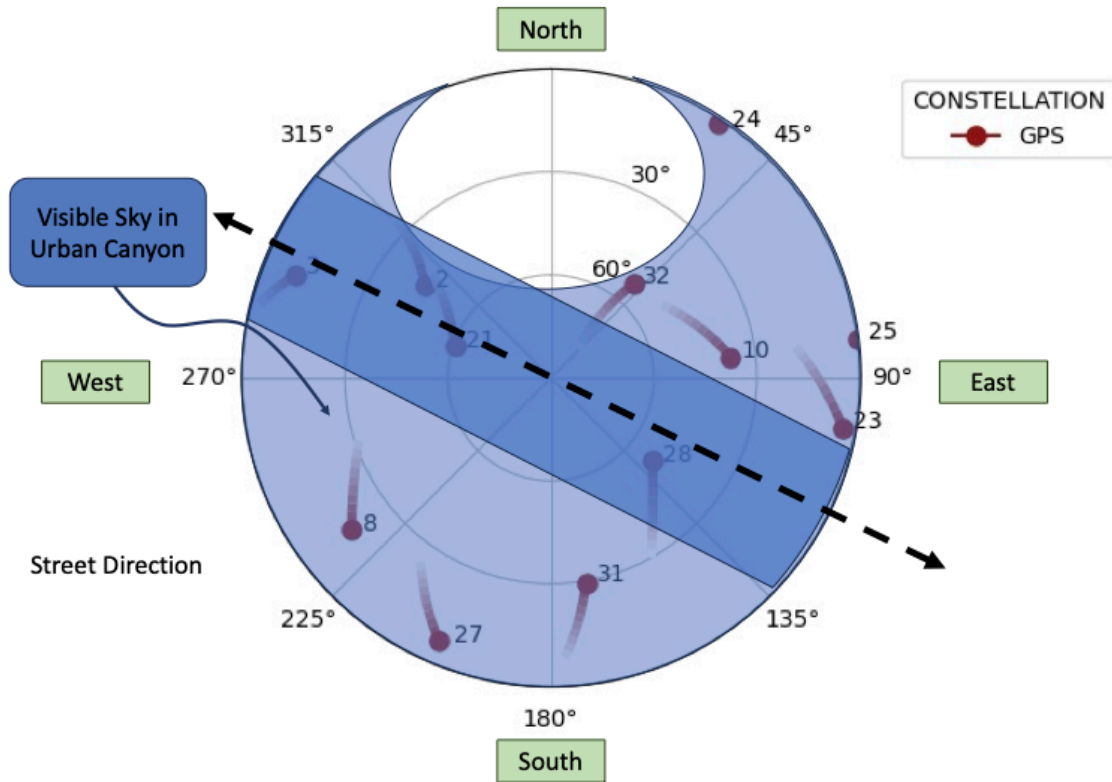
ANSWER

[Replace with your answer]

Now imagine that there were buildings around the receiver, forming an urban canyon, as shown below



These buildings will block the satellite signals. If a signal does reach the receiver, it will have bounced on other buildings before reaching the receiver. So, the measured pseudorange will have large errors (The technical term is **non-line-of-sight bias** or **NLOS bias**) Suppose we can correctly remove all the signals that are blocked or degraded from the buildings. The only remaining visible GPS satellites are 2, 3, 21, 28 (left to right) in the figure below.



3.5 [4 points]

Using your function, `compute_dop_matrix(az_arr, el_arr)`, output the **new DOP matrix** with only the four available satellites visible from within the urban canyon. Briefly comment on the difference you see in the DOP matrix. Specifically, what has changed in the diagonal components (i.e., EDOP, NDOP, VDOP, and TDOP) and the off-diagonal components?

ANSWER

[Replace with your answer]

Deliverable: The new DOP matrix $\tilde{\mathbf{H}}$ (as a heatmap plot) and brief (roughly 1-2 sentences) comments to the questions.

3.6 [3 points]

Using your function, `compute_pdop(H)`, what is the **PDOP** within the urban canyon. Is it higher or lower than before? Why?

ANSWER

[Replace with your answer]

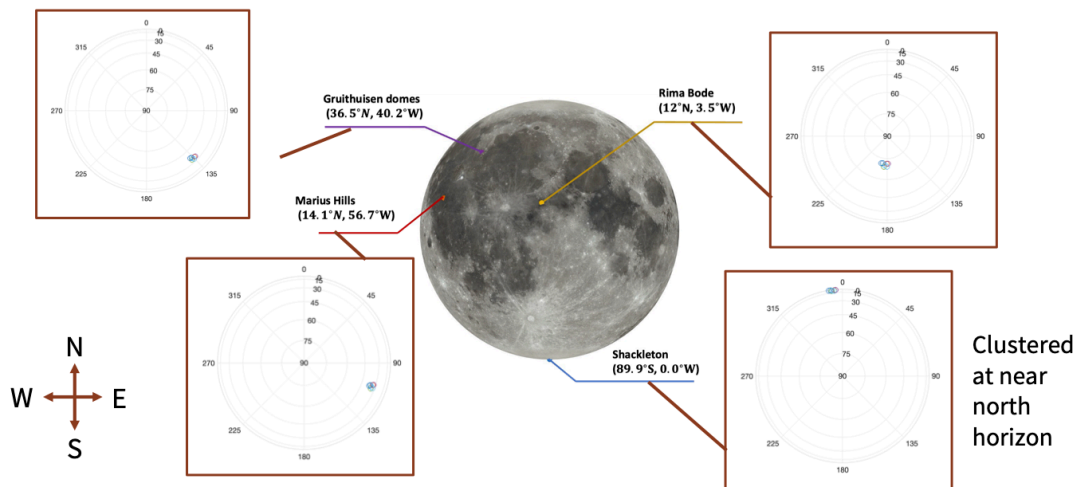
Deliverable: The new PDOP and a brief explanation as to why the PDOP has changed in the urban setting (roughly 1-2 sentences).

There is an ongoing effort to develop a receiver that can track weak GPS signals near the Moon (Example: LuGRE Mission). Figure below shows the skyplot of the GPS satellites at different locations on the Moon, where the GPS satellite directions are clustered in the lunar user's sky. In this bonus problem, we will think about the challenges of using GPS signals on the Moon by computing the DOP values.

We are provided with four CSV files on Canvas for the following lunar surface locations:

1. GruithuisenDomes: hw2_p3_bonus_Shackleton.csv
2. MariusHills: hw2_p3_bonus_MariusHills.csv
3. RimaBode: hw2_p3_bonus_RimaBode.csv
4. Shackleton: hw2_p3_bonus_GruithuisenDomes.csv

Each file contains the azimuth and elevation of the GPS satellites that are visible from the 4 locations on the lunar surface shown in the figure below (These values were computed using the GPS satellite positions and lunar position on August 1st, 2022)



Hint: The column labels of the azimuth and elevation are the same as the data we provided for Problem 3. If you are using gnsslibpy, you can load the csv files as `navdata` using the following function. Then, you can use the same function you implemented in Problem 3 to compute the DOP matrices.

```
import gnss_lib_py as glp
```

```
data_sh = glp.NavData(csv_path="hw2_p3_bonus_Shackleton.csv")
data_mh = glp.NavData(csv_path="hw2_p3_bonus_MariusHills.csv")
data_rb = glp.NavData(csv_path="hw2_p3_bonus_RimaBode.csv")
data_gd = glp.NavData(csv_path="hw2_p3_bonus_GruithuisenDomes.csv")
```

3.7 Bonus [1 point]

Using your function, `compute_dop_matrix(az_arr, el_arr)` and `compute_pdop(H)`, output the DOP matrix and PDOP value for each of the surface locations. At which surface location do you observe the **largest (worst) PDOP**? At what surface location do you see the **smallest (best) PDOP**?

ANSWER

[Replace with your answer]

Deliverable: 4 DOP Matrix plots and PDOP values. Largest/smallest DOP site

3.8 Bonus [1 point] For each of the 4 locations, compute the **Time Dilution of Precision (TDOP)** values. How does it compare with PDOP? (Is it significantly larger, significantly smaller, or nearly the same?) Why do you think this happens?

ANSWER

[Replace with your answer]

Deliverable: Comparison between PDOP and TDOP. 1-2 sentences describing the reason

3.9 Bonus [1 point] Consider navigating a user at **Shackleton** (which is located near the south pole). If you can add one more navigation satellite (a lunar GPS satellite!) anywhere in the sky to improve the PDOP (at this particular time epoch), where do you think would be a good location to add the new navigation satellite? Why do you think that would be a good location?

You do not have to compute the optimal location, nor compute the new DOP values. Also, You do not have to consider about the feasibility of the orbits.

ANSWER

[Replace with your answer]

Deliverable: A new satellite location that will lower PDOP. 1-2 sentences describing your reasoning

Self-care

Oftentimes, when we think of well-being, we might focus on the absence of illness or negative emotions. But, in addition to preventing ill health, we may further benefit by actively promoting flourishing in our lives. Reflect on this illuminating quote from Prof. Lynn Soots:

“Flourishing is the product of the pursuit and engagement of an authentic life that brings inner joy and happiness through meeting goals, being connected with life passions, and relishing in accomplishments through the peaks and valleys of life.”

What is 1 thing you have recently done (or 1 thing you will set an intention to do) which has helped you flourish in this moment of your life, whether it’s a “peak” or “valley”? Describe your thoughts in a few sentences. Alternatively, feel free to be creative in your form of expression, including but not limited to, a photo, video, or sketch.

ANSWER

[Replace with your answer]